

Niv and Mach-Nix for Nix Python

Rohit Goswami · 2020-08-26 · tools · nix · workflow · python

Short post on using `mach-nix` with `niv`.

Background

In previous posts, there was a [discussion on a ground up approach to adding packages](#) which aren't on the core `nixpkgs` channels using GitHub or PyPi sources. However, this lacked a way to do so programmatically, and also a way to convert existing python projects.

Python Dependency Management

This time, instead of the more pedagogical approach of building packages from PyPi or GitHub, we will use overlays and the excellent `mach-nix` to speed up the process. We will continue to use `niv`.

```
niv init
niv update nixpkgs -b nixpkgs-unstable
```

To leverage `mach-nix` we will simply need the following setup to work with `niv`.

```
let
  sources = import ./nix/sources.nix;
  pkgs = import sources.nixpkgs { };
  inherit (pkgs.lib) optional optionals;
  mach-nix = import (builtins.fetchGit {
    url = "https://github.com/DavHau/mach-nix/";
    ref = "refs/tags/3.1.1";
  }) {
    pkgs = pkgs;

    # optionally specify the python version
    # python = "python38";

    # optionally update pypi data revision from https://github.com/DavHau/pypi-deps-db
    # pypiDataRev = "some_revision";
    # pypiDataSha256 = "some_sha256";
  };
  customPython = mach-nix.mkPython {
    requirements = ''
      copier
      pytest
    '';
    providers = {
      _default = "nixpkgs,wheel,sdist";
      pytest = "nixpkgs";
    };
    pkgs = pkgs;
  };
in pkgs.mkShell { buildInputs = with pkgs; [ customPython ]; }
```

Note that we have essentially written out a `requirements.txt` and can actually pass a path there instead as well. The key point to make it work with `niv` is the `pkgs` parameter. To use the older method of overriding parts of the setup, we can use the `overrides_pre` hook as shown below:

```
let
  sources = import ./prjSource/nix/sources.nix;
  pkgs = import sources.nixpkgs { };
  inherit (pkgs.lib) optional optionals;
  mach-nix = import (builtins.fetchGit {
    url = "https://github.com/DavHau/mach-nix/";
    ref = "refs/tags/3.1.1";
  }) {
    pkgs = pkgs;

    # optionally specify the python version
    # python = "python38";

    # optionally update pypi data revision from https://github.com/DavHau/pypi-deps-db
    # pypiDataRev = "some_revision";
    # pypiDataSha256 = "some_sha256";
  };
  customPython = mach-nix.mkPython {
    requirements = ''
      copier
      pytest
      f90wrap
    '';
    providers = {
      _default = "nixpkgs,wheel,sdist";
      pytest = "nixpkgs";
    };
    overrides_pre = [
      (pythonSelf: pythonSuper: {
        pytest = pythonSuper.pytest.overrideAttrs (oldAttrs: {
          doCheck = false;
          doInstallCheck = false;
        });
        f90wrap = pythonSelf.buildPythonPackage rec {
          pname = "f90wrap";
          version = "0.2.3";
          src = pkgs.fetchFromGitHub {
            owner = "jameskermode";
            repo = "f90wrap";
            rev = "master";
            sha256 = "0d06nal4xzg8vv6sjdbmg2n88a8h8df5ajam72445mhzk08yin23";
          };
        };
        buildInputs = with pkgs; [ gfortran stdenv ];
        propagatedBuildInputs = with pythonSelf; [
          setuptools
          setuptools-git
          wheel
          numpy
        ];
        preConfigure = ''
          export F90=${pkgs.gfortran}/bin/gfortran
        '';
        doCheck = false;
        doInstallCheck = false;
      });
    ];
  };
  in pkgs.mkShell { buildInputs = with pkgs; [ customPython ]; }
```

We can also pull in overrides from `poetry2nix` with `overrides_post` as described [here](#).

Conclusion

With the completion of this final remaining hurdle, `nix` is now fully realized as a `python` management system. At this point the “only” thing remaining is to find an optimal way of using `nix` for setting up reusable data science and scientific computing projects.